

전자전기컴퓨터설계실험II

실험 후 보고서

LAB4

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 10월 16일
대상	LED와 버튼 및 FPGA 조합회로
검증 환경	VS Code / Vivado / STM32 / FPGA

1. 실험 목적과 관련 이론

실험 목적

NUCLEO-G474RE의 사용자 버튼을 읽고 LED의 상태 변경 간격을 프로그램으로 조절한다. 과제 1에서는 PA5 내장 LED를 사용하고, 과제 2에서는 같은 동작을 PB0 외부 LED에 적용한다. 버튼을 누를 때마다 간격이 1초, 2초, 4초, 8초, 1초로 순환하도록 설계한다.

GPIO 입력과 출력

GPIO는 핀을 디지털 입력 또는 출력으로 사용하는 기능이다. 수업 프로젝트에서 PC13은 사용자 버튼, PA5는 내장 LED, PB0는 외부 출력으로 설정되어 있다. PA5는 HIGH일 때 켜지고 LOW일 때 꺼진다.

HAL_GPIO_ReadPin()으로 버튼 상태를 읽고 HAL_GPIO_TogglePin()으로 LED 출력을 반전한다. GPIO_PIN_SET과 GPIO_PIN_RESET은 각각 HIGH와 LOW를 뜻한다.

폴링과 시간 기준

메인 루프에서 polling_routine()을 반복 호출한다. HAL_GetTick()의 밀리초 값을 읽어 마지막 출력 변경 시각과 비교한다. HAL_Delay()로 실행을 멈추지 않으므로 점멸 간격을 기다리는 동안에도 버튼 입력을 확인할 수 있다.

임베디드 시스템 교재는 LED 점멸, 버튼 입력, 디바운싱을 입출력, 타이머 설계의 연결된 주제로 다룬다. 본 실험도 입력 검출, 시간 판단, 출력 변경으로 기능을 나누어 구성한다.

간격과 전체 주기

상태 변경 간격을 Δt라고 하면 켜짐과 꺼짐이 각각 Δt 동안 유지된다. 따라서 전체 점멸 주기는 $T = 2\Delta t$이고 주파수는 $f = 1/(2\Delta t)$이다. 과제의 1초 간격은 전체 주기 2초를 의미한다.

버튼의 누른 순간 검출

현재 버튼 값이 HIGH이고 이전 값이 LOW일 때만 새 입력으로 처리한다. 이전 값을 static 변수에 보관하므로 버튼을 누르고 있는 동안에는 간격이 반복해서 증가하지 않는다.

첫 입력은 즉시 허용하고, 이후 마지막 유효 입력에서 200ms가 지나지 않은 반복 검출은 무시한다. 이 처리는 수업 예제의 간단한 접점 바운스 억제 방식이다. 200ms 이상 안정된 입력을 보장하는 필터와는 다르므로 실제 버튼에서 중복 입력 여부를 확인한다.

FPGA 조합회로의 원리

곱셈기와 BCD 변환기는 클록 없이 입력 조합으로 출력이 정해진다. 곱셈기는 0~15의 두 수를 곱해 최대 225를 출력하므로 8비트가 필요하다. 두 자리 BCD의 정상 범위는 0~99이므로 변환 결과는 7비트로 표현할 수 있다.

검증 방법

STM32는 실제 버튼 조작과 LED 관찰 결과로 기능을 판단하였다. FPGA는 테스트벤치의 전수 검사 로그, Vivado 파형, 구현 완료 화면, 보드 사진을 함께 비교하였다. 기능 검증과 정밀 시간 측정은 구분하여 결과를 기록하였다.

실험 장치와 개발 환경

항목	사용 내용
보드	NUCLEO-G474RE

입력	PC13 사용자 버튼 B1
과제 1 출력	PA5 내장 LED
과제 2 출력	PB0 외부 LED
외부 부품	LED, 330 Ω~1 kΩ 저항, 브레드보드, 점퍼선
연결	ST-LINK USB 데이터 케이블
소프트웨어	VS Code, STM32 HAL, CMake, Ninja, ARM GCC·GDB

주요 도구 버전

- 컴파일러: Arm GNU Toolchain 14.3.Rel1
- 빌드 도구: CMake 4.4.0과 Ninja 1.13.2
- ST-LINK GDB Server: 7.14.0
- STM32CubeProgrammer: 2.23.0

2. 과제 1 내장 LED 간격 변경

설계 방법과 핵심 코드

시작 간격은 1000 ms이다. 버튼을 누를 때 간격을 두 배로 늘리고 8000 ms 다음에는 1000 ms로 되돌린다. 다음 코드는 실제 polling_routine()의 핵심 부분이다.

```
if (current == GPIO_PIN_SET
    && previous == GPIO_PIN_RESET
    && (press_seen == 0U
        ||  now_ms-last_press_ms  >=
200U))
{
    last_press_ms = now_ms;
    press_seen = 1U;
    interval_ms  =  (interval_ms  ==
8000U)
        ? 1000U : interval_ms * 2U;
    last_toggle_ms = now_ms;
}
previous = current;
if     (now_ms-last_toggle_ms     >=
interval_ms)
{
    last_toggle_ms = now_ms;
    HAL_GPIO_TogglePin(
        LED_ece2_GPIO_Port,
LED_ece2_Pin);
}
```

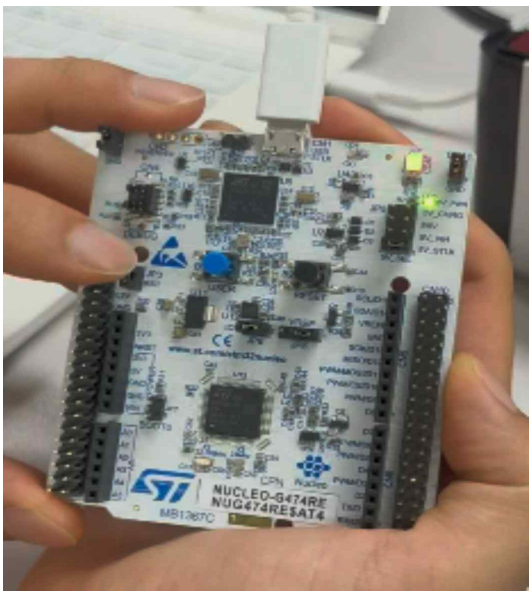
interval_ms와 이전 입력 값은 static 변수로 보관한다. 버튼을 누를 때 시간 기준도 갱신하므로 LED는 즉시 반전되지 않고 새로 정한 간격이 지난 후 반전된다.

예상 결과와 실제 관찰

검사 항목	예상	관찰 결과
초기 동작	1초 간격	정상 점멸
버튼 순차 입력	2→4→8→1초	순환 성공
버튼 입력 없음	선택 간격 유지	동작 성공
실험 중 이상	중복·멈춤 없음	특이사항 없음

간격 변화와 반복 점멸은 기능적으로 확인하였다. 별도의 시간 계측값은 기록하지 않았으므로 실제 간격의 평균, 표준편차, 상대 오차는 산출하지 않았다.

실제 회로 구성

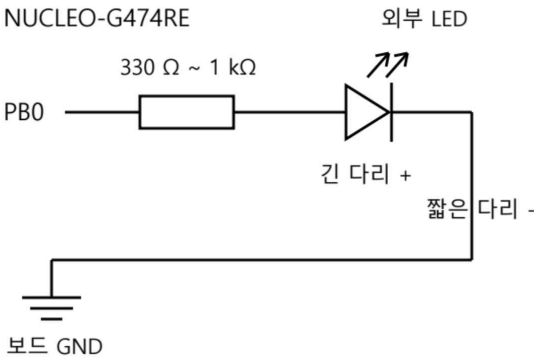


외부 LED를 사용하지 않고 내부 LED를 사용했으며, 버튼도 내장된 버튼을 사용했다.

3. 과제 2 외부 LED 동작

회로 구조와 코드 변경

버튼 처리와 시간 제어는 과제 1과 같고, 출력만 PB0로 변경하였다. PB0에서 저항을 거쳐 LED의 긴 다리로 연결하고 짧은 다리는 GND에 연결하는 구조이다.



직렬 저항은 LED 전류를 제한한다. 저항 전류의 근삿값은 I=(VOUT−VF)/R이다. 이는 저항에 옴의 법칙을 적용한 식이며, LED 자체의 전압·전류 관계를 선형으로 가정한 것은 아니다.

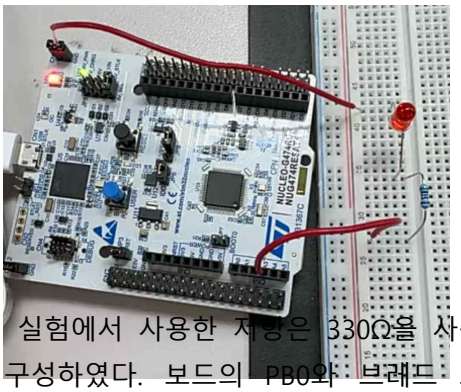
```
#if defined(STM32_0_EXTERNAL_LED_PB0)
HAL_GPIO_TogglePin(
    GPIO_OUT1_ece2_GPIO_Port,
    GPIO_OUT1_ece2_Pin);
#else
HAL_GPIO_TogglePin(
    LED_ece2_GPIO_Port,
LED_ece2_Pin);
#endif
```

과제별 빌드와 실제 결과

설정	출력	결과
Debug	PA5 내장 LED	과제 1 성공
ExternalLED	PB0 외부 LED	과제 2 성공

외부 LED에서도 1→2→4→8→1초의 간격 순환을 확인하였다. 내장 LED와 같은 입력 처리 논리가 출력 핀 변경 후에도 동작하였으며, 실험 중 별도 실패나 특이사항은 없었다.

실제 회로 구성



실험에서 사용한 저항은 330Ω을 사용하여 구성하였다. 보드의 PB0와 브레드 보드를 점퍼선을 이용하여 연결하고 330Ω의 저항과 LED를 직렬로 연결하고 다시 점퍼선으로 브레드 보드와 보드의 GND를 연결하여 회로를 구성했다.

4. 소스 구성과 GitHub 주소

파일	역할
ece2_474_routine.c	버튼 검출, 간격 갱신, LED 반전
main.c / main.h	초기화, 메인 루프, 핀 정의
CMakeLists.txt	외부 LED 빌드 옵션
CMakePresets.json	Debug / ExternalLED 설정
ece2_474.code-workspace	Configure, Build, 디버그 연결

저장소와 제출 버전

- GITHUB저장소: https://github.com/thechef030/STM32_0
- 브랜치: stm32-0
- 코드 태그: v1.1.0
- 태그가 가리키는 전체 커밋 해시: `14caa8a56b244905436d1b7e1efb89844341fffa`

-레포트와 실험 결과를 업로드한 후의 전체 커밋 해시:

```
5236de07bbf1eafb08234acc65a9b06fa2896041
```

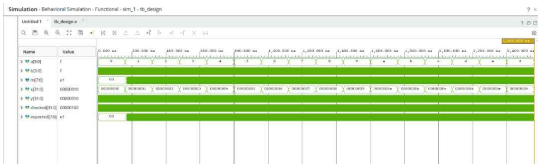
5. 실험 1 4비트 곱셈기

설계 목표와 회로 구조

a[3:0]와 b[3:0]를 DIP로 입력하고 m[7:0]를 LED로 출력하였다. 최상위 mult_4bit가 multiply_unsigned 모듈을 연결한다. 내부 곱셈은 조합 논리이며 최대 출력은 15×15=225이다.[1]

```
module multiply_unsigned #(
    parameter WIDTH=4
)(
    input wire [WIDTH-1:0] a, b,
    output     wire     [2*WIDTH-1:0] product
);
    assign product = a * b;
endmodule
```

Vivado 기능 검증



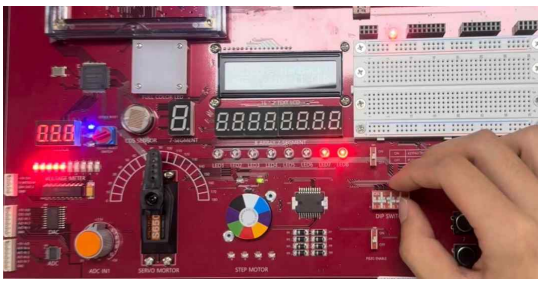
16×16=256개 입력 조합을 10 ns 간격으로 인가하고 예상 곱과 비교하였다. 로그에서 PASS multiplier: 256 input pairs를 확인하였다. 마지막 입력 a=b=F(16진수)일 때 m=E1이며, 10진수로 15×15=225와 일치한다.

(시간 범위 0~2560 ns)

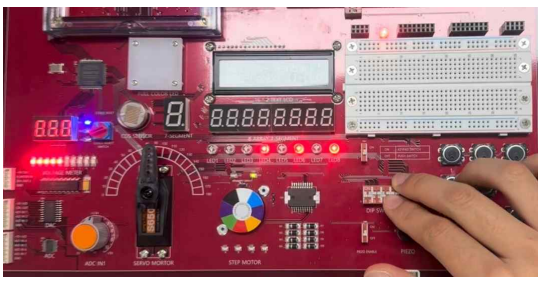
Name	Constraints	Status
✓ synth_1	constrs_1	synth_design Complete!
✓ impl_1	constrs_1	write_bitstream Complete!

사용 자원은 LUT 16개, FF·BRAM·DSP 0개이며 Failed Routes는 0이다. 화면의 WNS·TNS는 NA이므로 별도의 타이밍 여유가 검증되었다고 해석하지 않는다. Methodology 경고 2건이 표시되어 있다.

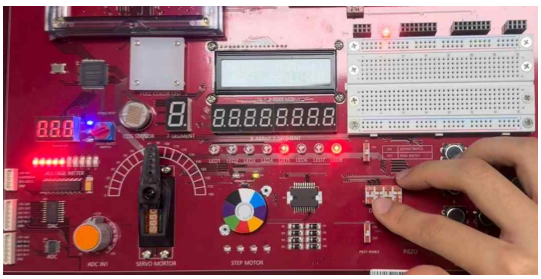
보드 관찰과 예상값 비교



- LED7·LED8 점등: $00000011_2 = 3$.



- LED4·LED6·LED8 점등: $00010101_2 = 21$.



- LED5·LED8 점등: $00001001_2 = 9$.

LED1에서 LED8 순서로 m[7]에서 m[0]를 읽는다. 사진은 DIP를 조작했을 때 LED 출력 패턴이 바뀌는 것을 보여준다.

수학적 시험값	예상 출력
1×3	$00000011_2 = 3$
3×7	$00010101_2 = 21$
3×3	$00001001_2 = 9$
15×15	$11100001_2 = 225$

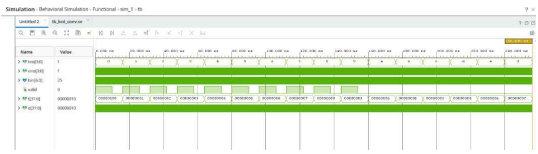
6. 실험 2 BCD 이진수 변환

설계 목표와 핵심 코드

DIP 상위 4비트로 십의 자리 ten, 하위 4비트로 일의 자리 one을 입력하였다. 출력은 binary=10×ten+one이며, 두 입력이 모두 0~9일 때만 valid=1이다.[1]

```
module bcd_to_binary(
    input wire [3:0] ten, one,
    output wire [6:0] binary,
    output wire valid
);
    assign binary = ten * 7'd10 + one;
    assign valid = (ten <= 9)
                && (one <= 9);
endmodule
```

Vivado 기능 검증



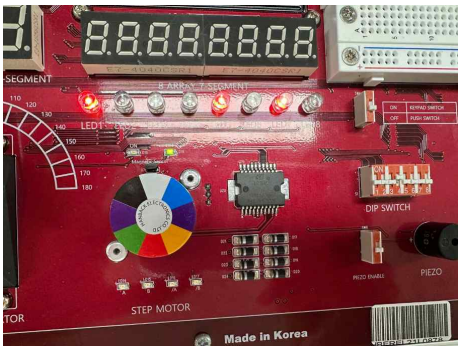
테스트벤치는 256개 조합의 valid를 검사하고, 정상 BCD 100개에서 변환값을 비교하였다. 로그에 PASS BCD: 100 valid and 156 invalid inputs가 출력되었다. 마지막 ten=F, one=F는 BCD 범위를 벗어나 valid=0이다. bin=25(16진수)는 165의 하위 7비트인 37이며 정상 BCD 변환값이 아니다. 이 회로는 무효 입력에서도 산술 출력을 계산하므로 valid와 함께 해석해야 한다.

(시간 범위 0~256 ns)

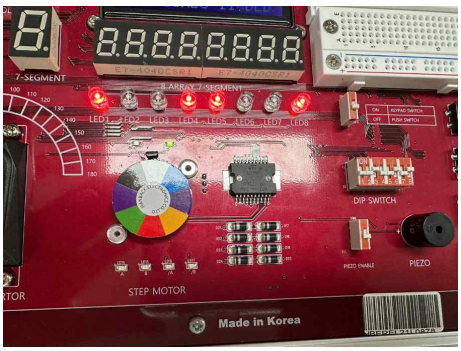
Name	Constraints	Status
✓ synth_1	constrs_1	synth_design Complete!
✓ impl_1	constrs_1	write_bitstream Complete!

LUT 7개, FF·BRAM·DSP 0개, Failed Routes=0으로 비트스트림 생성이 완료되었다. Implementation 경고 3건이 표시되며, NA로 표시된 타이밍 수치를 합격 수치로 사용하지 않았다.

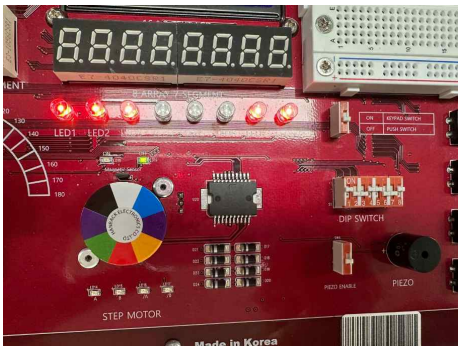
보드 출력과 유효 신호



- valid=1, bin=0001010$_2$ = 10.



- valid=1, bin=0011001$_2$ = 25.



- valid=1, bin=1100011$_2$ = 99.

LED1은 valid이고 LED2~LED8이 bin[6:0]이다. 따라서 LED1까지 결과값에 포함하면 128이 더해져 잘못 읽게 된다. 세 사진에서 LED1이 켜져 있고 나머지 7개 LED는 각각 10, 25, 99를 나타낸다.

BCD 입력	예상 이진수	사진 출력
0001 0000	0001010 (10)	10
0010 0101	0011001 (25)	25

BCD 입력	예상 이진수	사진 출력
1001 1001	1100011 (99)	99

DIP의 십·일 자리 입력과 LED 출력 비교. 10진수 각 자리를 4비트로 입력한 BCD와 결과 이진수의 비트열은 서로 다르다.

7. 참고문헌

[1] 전전설2 LAB4 교안